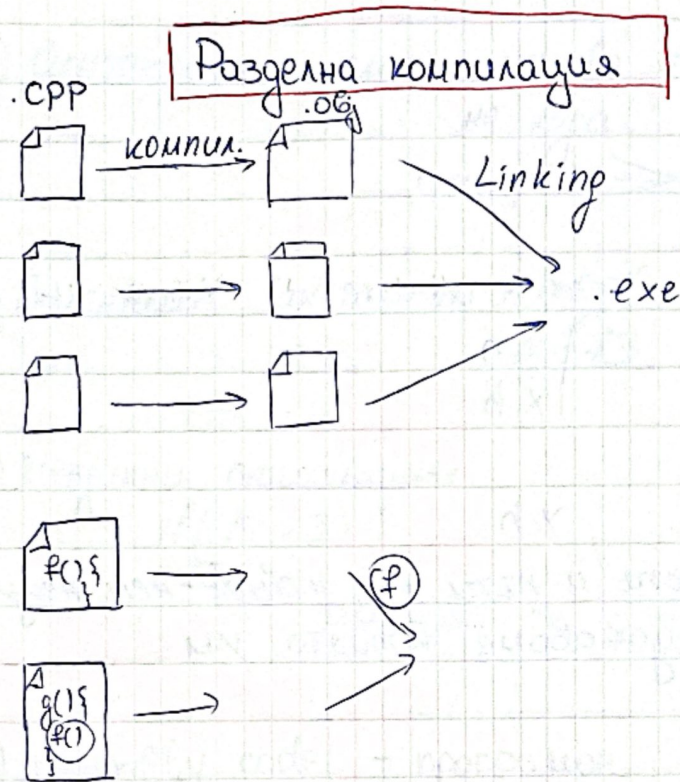


26.03.2024г.

02.04.2024г. и 09.04.2024г. - лекциите ще се водят от
Георги Йосифов



формално декларация // обещание, че функцията ще се намери при linking
Ако функцията не съществува, дава linking грешка.

Ако променим един .cpp файл, другите няма да се променят и ще се ползват .obj файловете им

Forward декларации ги слагаме в .h файлове
Когато искаме да го ползваме .cpp

include

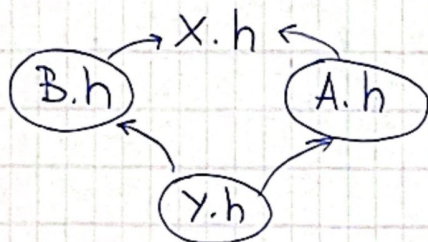
клас X

X.h

→ декларациите и обещанията за функции

X.cpp

→ имплементациите



Така имаме:

x.h
x.h

y.h

#pragma once - колкото и пъти да include-нем нещо
в унищожаване копията им

{ A.cpp, B.cpp ... }

① Препроцесор - "обработка на стрингове"

`#include ... "A.txt"` - замества се със
съдържанието на
A.txt

→ Макроси - нещо като мини функции, които ни
казват заместим код с друг код

`#define A 73` // Всяко
A се заменя с 73

~~#~~ 73 // стринго обработка

`int a = 73;` // Това е различно
a; // заема памет в стека

inline f()

не използва стековата рамка

2) Синтактичен анализ → минава и през синтактичния анализ на кора

пример: ~~a++~~

3) Семантичен анализ → ~~A obj~~
obj = 73

4) Метринни оптимизации
if(10 > 3)

замества се с талото на if-а

5) Assembly code - програмен език

6) Машинен код - 0 и 1 - компилирания код
компиляционен процес

7) Линкване

имаме .obj файл

8) Оптимизации
сри-depend оптимизации

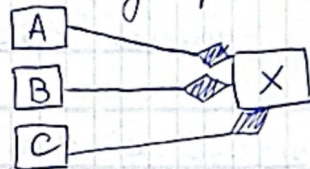
Композиция и агрегация

- Взаимотношения между обекти.

1) Композиция

```
class X {  
    A obj1;  
    B obj2;  
    C obj3;  
};
```

UML диаграми - имае квадратче, за всеки class



▬ - композиция
→ - наследяване

⚠ Жизненият цикъл на A, B и C се контролира от X.

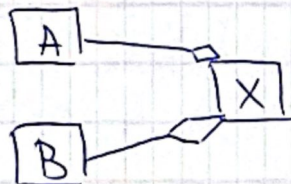
// Конструктора на X създава A, B, C

Деструктора на X унищожава A, B, C

2) Агрегация

```
class Y {  
    A* a; // указатели или референции, към  
    B& b; обекти, които са самостоятелни  
};
```

UML:



◊ - агрегация

⚠ A, B могат да живеят извън рамките на Y

// Конструктор

Y(A* ptr1, B* ptr2) : a(ptr1), b(*ptr2)

~Y // не извиква деструкторите на A и B


```
class Z {
```

```
  A obj;  
  B* ptr;  
  C obj3;
```

композиция

// заради B*

Z() // избира констр. A() и C()
ptr = new B() - зареждане памет
избира констр. B()

~Z()

delete ptr;
// ~B(), ~C() и ~A()

Config C; // глобален обект // пример за агрегация
App1 a1(C);
a1.run();

App2 a2(C);
a2.run();

```
App1 ap1  
{  
  config c;  
  ap1.set_config(c);  
}
```

config живее до края на scope

ap1.run();
// лош пример за агрегация

заради / Event - клас ← за събесерване
• data Date
• Time Time
• начален час - краен час
• име

началния час < краен час - Валидация за класа Event

ТЕМА 6:

Копиране на обекти

```
struct A {  
  char ch1; :4  
  char ch2; :4  
}
```

използва се 4 бита
1 байт A obj1;
A obj2; } копиране
още 4 бита

```
struct X {  
  char ch // char ch:8
```

• Копиращ конструктор

- приема обект от същия клас
- текущия става негово копие

```
// конструктора :  
A (const A &)
```

⚠ Ако не го създадем и напишем копиращия конструктор, то компилатора го създава.

```
class X {  
  int a;  
  char ch;  
  A obj;  
  B obj2;  
};  
X obj;  
X copy(obj);
```

Копиращия конструктор на X извиква копиращите конструктори на A и B, а на a и ch извиква местото им в паметта (мемори).


```
struct Y {
```

```
  A obj1;
```

```
  B obj2;
```

```
};
```

```
Y (const Y & other):
```

```
  obj1 (other.obj1)
```

```
  obj2 (other.obj2)
```



Как се извиква ?

```
A obj1;
```

```
A obj2 (obj1); → K.K
```

Ако имаме : $f(A\ obj1);$ // по копие
 $g(A\ \&\ obj1);$

$f(obj1) \rightarrow K.K.$

$g(obj1)$ // нищо няма да се извика, защото е по референция

2) Оператор = (Оператор за присвояване)

→ Приема обект от същия тип

и текущия става негово копие

→ Текущият обект е съществувал преди това.

```
A obj1;
```

```
A obj2;
```

```
obj1 = obj2;
```

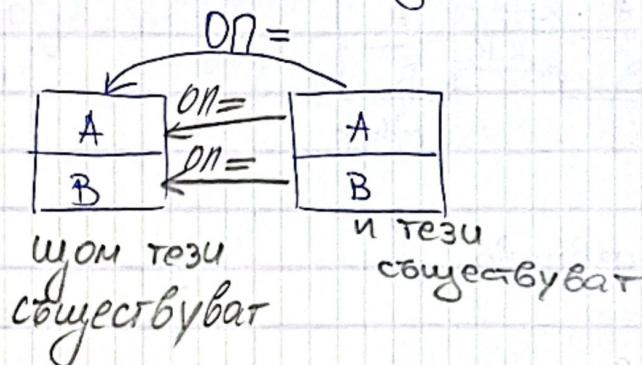
// Всички данни от obj2 копира в obj1

Операторът =:

- изчислява текущите данни
- копира // копирацията конструктор има само това

Ако не го разпишем, компилаторът създава такъв.

```
class X {  
    A obj1;  
    B obj2;  
}
```



X& operator = (const X& other)

→ връща резултат, връща левия аргумент

$$\underbrace{a = b}_{a}$$
$$(a = (b = (c = x)))$$
$$\underbrace{\underbrace{c}_{b}}_a$$

obj1 = other.obj1;
obj2 = other.obj2;
return *this;



X obj1;
X obj2 = obj1; // Извиква копиращия
конструктор, защото
obj2 не съществува

X O1;

X O2;

O1 = O2;

← изтриват се данните от O1
← копират се данните от O2

X 01; // проблем
 01=01; // вече са изтрети ранните за 01
 // трябва да се погрижим за този проблем;
 // трябва да добавим проверка
 // проверяваме дали са различни адресите
 if (this != &other)

```

class X {
  A obj1;
  B obj2;
  C obj3;
};
  
```

// Копиращ конструктор
 X (const X& other)
 това са копиращи конструктори
 { obj1 (other.obj1);
 obj2 (other.obj2);
 obj3 (other.obj3);
 }
 }

Ако пропуснем това ще се извика default конструктори

```

X& operator = (const X& other)
{
  if (this != &other)
  {
    obj1 = other.obj1;
    obj2 = other.obj2;
    obj3 = other.obj3;
  }
  return *this;
}
  
```

отговарящото
 копие е

Задатък / Отговорение В копиращия конструктор на X, A, B, C
 и в началото на оператор = на X, A, B, C.
 Какво ще се отпечата на екрана?

```

class X {
  A a1;
  A a2;
  B obj1;
  C obj2;
};
  
```



```

{
X obj;
X obj2 = obj;
obj = obj2;
X* ptr = &obj; // тук нищо не се отпегатва
// К.К на X
g(ptr); // нищо не се отпегатва
}

```

$\{ (X \text{ obj}) \} \rightarrow X(), \sim C(), \sim B, 2x \sim A()$
 $\{ (X^* \text{ ptr}) \} \rightarrow g(X^* \text{ ptr})$

C - копиращ конструктор и оператор =
 m - конструктори и деструктори

К.К. А, К.К. А, К.К. В, К.К. С, К.К. X

оп = X, оп = А, оп = А, оп = В, оп = С

X-нищо ако отпегатването е в края

К.К. А, К.К. А, К.К. В, К.К. С, К.К. X
 X-нищо

A(), A(), B(), C(), X() - default конструктори

2x ~X(), ~C(), ~B(), ~A(), ~A()

A createA()
 return A(); def. конструктор

A obj = createA() → К.К конструктор

отговор: Извиква и принтира default - конструктор, но не отпегатва К.К. констр.

Използва RVO (return value optimisation) // не отпегатва К.К

A create A()

A obj;

return obj;

named
NRVO

обектът не е създаден в return.

⚠ Проблем при генерираните от ~~не~~ компилатора
Оператор = и копир. конструктор
→ При работа с динамична памет.

```
struct T {
```

```
    char *str;
```

```
    int n
```

```
    T() = {
```

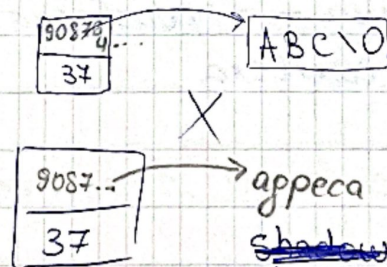
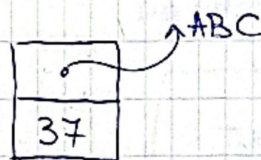
```
        str = new char [7];
```

```
    }
```

```
    ~T() { delete str; }
```

```
Test t1;
```

```
Test t2(t1);
```

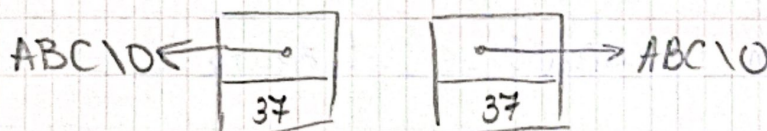


адреса на ~~на~~ горното
~~shadow~~ copy - не го искаме
shallow

искаме

deep copy

// да имат една и съща стойност, но
различни адреси



Default-конструктор се създава винаги, а
default-копиращ конструктор се създава, ако не
забраним.

За да решим проблема, когато имаме динамична памет,
ние разписваме { К.К. // default-ния конструктор
изглежда

Big 4
големата
четворка { оператор =
деструктор
трябва да разпишем и
default-конструктор

Ако нямаме динамична памет не ги пишем

К.К - копира
деструктор - true
оп = - true
- копира

→ изнасяме ги в други
функции
→ copy From
→ Free

Пример за изпит:

за копиране на масив (статичен)
правим го в struct и използваме
копиране в структурата

Запис за завършил студент - клас

- име - произволна дължина
- масив от оценки → произволна дължина
- цитат = 30 символа

// 2 динамични масива и 1 статичен

Ако # направим = delete // забраняваме ги, за да не
се правят копия

за изпит и контролно, ако нямам време да ги разпи-
шем

При copy From не използваме set-ърите


```
struct A1 {  
    char str[10];
```

```
    char *get() const // Няма да се компилира  
    { return str; }  
};  
добавене const
```

```
A2 {  
    char *str;
```

```
    char *get() const // компилира се  
    но добавене const
```